# Comparison of Elman and Jordan Networks for Next-Character Prediction

**Course:** Neural Networks and Deep Learning

**Assignment:** Assignment 2

**Student:** Saghar Kheradmand

**University:** Shiraz University

# Abstract

This report presents a character-level implementation and comparison of two classical recurrent architectures, the Elman Network and the Jordan Network, on the Tiny Shakespeare dataset. To reduce training time, 30% of the dataset was used. The central objective was to investigate how the recurrent feedback mechanism affects learning quality: Elman feeds the previous hidden state into the next time step, whereas Jordan feeds back the previous model output.

Both models were trained under identical experimental settings and were evaluated using training and validation cross-entropy loss, character-level accuracy, perplexity, convergence behavior, training stability, and qualitative text generation. The results consistently favored the Elman architecture. At the end of epoch 15, Elman achieved a validation accuracy of 47.65% and a validation perplexity of 6.29, while Jordan achieved 36.10% and 9.16, respectively. The best validation accuracies were 47.77% for Elman and 36.23% for Jordan, both at epoch 14.

Text generation experiments were also performed using the same seed, “ROMEO:\n”, at temperatures 0.4, 0.8, and 1.2. Elman generally produced more coherent and natural-looking output, while Jordan showed more repetition, malformed words, and instability, especially as temperature increased. Overall, the experiment indicates that hidden-state feedback was more effective than output-distribution feedback for this dataset and configuration.


**Implementation file:** p2_completed.py

**Dataset used:** tiny-shakespeare(1).txt — 30% of the data

# Table of Contents

# 1. Introduction

The main purpose of this assignment is to implement and compare two classical recurrent neural network architectures—Elman and Jordan—for next-character prediction. Next-character prediction is a fundamental language-modeling problem because the model must use preceding characters as temporal context in order to estimate the character that follows.

Text is sequential data, so character order is essential. In dramatic text such as Shakespeare, character names, colons, line breaks, dialogue patterns, spaces, and punctuation appear in recurring structures. A useful recurrent model must therefore preserve information from earlier time steps and use that information to predict subsequent characters.

The Tiny Shakespeare dataset was used because it contains Shakespeare-style dramatic text and is well suited to sequence modeling and text-generation experiments. The corpus was processed at the character level: every letter, space, punctuation mark, and newline was treated as an individual token.

The key distinction between the two architectures is the source of recurrent feedback. Elman uses the previous hidden state as internal memory, whereas Jordan uses the previous output distribution. The experiment investigates which feedback mechanism is more effective for learning the sequential patterns present in Tiny Shakespeare.

# 2. Objectives

The assignment is not limited to implementing two recurrent networks; it requires a controlled and fair comparison of their quantitative and qualitative behavior. Both models were therefore trained with the same settings so that observed differences could be attributed primarily to architecture and feedback design.

- Compare training and validation loss.
- Compare character-level prediction accuracy.
- Compare perplexity (PPL).
- Evaluate convergence speed and training stability.
- Generate text from the same seed for a fair qualitative comparison.
- Study how different sampling temperatures affect coherence, repetition, and diversity.

# 3. Dataset and Preprocessing

Only the first 30% of Tiny Shakespeare was used to reduce computational cost, which is consistent with the assignment specification allowing a subset of the corpus. The selected portion contained 334,618 characters and a vocabulary of 63 unique characters.

| Item | Value |
|---|---|
| Percentage of dataset used | 30% |
| Characters used | 334,618 |

| | |
|---|---|
| Vocabulary size | 63 |
| Training characters | 301,156 |
| Validation characters | 33,462 |

After loading the text and selecting the first 30%, all unique characters were extracted. Two mappings were constructed: stoi for converting a character to an integer index and itos for converting an index back to a character. The entire text was then encoded as a sequence of integer IDs.

The encoded sequence was split into 90% training data and 10% validation data. During batching, each input sequence contained 100 characters, and the target at every position was the next character in the sequence. This creates a standard character-level autoregressive prediction task.

# 4. Model Architectures

## 4.1 Elman Network

The Elman network is one of the simplest and best-known recurrent architectures. Its previous hidden state is passed to the next time step and acts as internal memory, allowing the model to retain information from preceding characters.

$h_t = \tanh(W_{xh} x_t + W_{hh} h_{(t-1)} + b)$

$y_t = W_{hy} h_t$

Because the recurrent memory is stored internally in the hidden state, Elman can preserve sequential context without directly feeding its previous prediction back as the next recurrent signal. In this experiment, that mechanism produced more stable learning of simple and medium-range character patterns.

## 4.2 Jordan Network

The Jordan network has a similar overall structure, but its feedback mechanism is different. Instead of using the previous hidden state, it uses the previous output distribution as recurrent context.

$h_t = \tanh(W_{xh} x_t + W_{yh} y_{(t-1)} + b)$

$y_t = W_{hy} h_t$

The previous prediction therefore acts as memory and directly influences the next state. This can be useful, but it also creates a potential error-propagation mechanism: if the model produces a poor prediction at one step, that imperfect output can influence later steps and make learning more difficult.

# 5. Experimental Configuration

Both models were trained with the same final configuration in p2_completed.py:

| Parameter | Value |
|---|---|
| Sequence length | 100 |
| Batch size | 64 |
| Embedding size | 64 |
| Hidden size | 128 |
| Epochs | 15 |

| | |
|---|---|
| Steps per epoch | 100 |
| Optimizer | Adam |
| Learning rate | 0.001 |
| Gradient clip | 1.0 |
| Device | CPU |
| Seed text | ROMEO:\n |
| Temperatures | 0.4, 0.8, 1.2 |

# 6. Evaluation Metrics

## 6.1 Cross-Entropy Loss

Cross-entropy loss was used because next-character prediction is a multi-class classification problem. At each time step, the model must choose among 63 vocabulary characters. The output represents a distribution over the vocabulary, and cross-entropy measures the discrepancy between that predicted distribution and the true next-character label. Lower loss indicates better prediction.

## 6.2 Character Accuracy

Character accuracy measures the percentage of positions for which the highest-probability predicted character matches the ground-truth next character. Higher accuracy means the model is more successful at directly selecting the correct next symbol.

Accuracy = Correct Predictions / Total Predictions

## 6.3 Perplexity

Perplexity is derived from the loss and is a common language-modeling metric. It reflects how uncertain the model is when predicting the next character. Lower perplexity means less uncertainty.

PPL = exp(loss)

# 7. Final Numerical Results

The following table reports the final values at the end of epoch 15:

| Model | Train Loss | Train Acc | Train PPL | Val Loss | Val Acc | Val PPL |
|---|---|---|---|---|---|---|
| Elman | 1.5667 | 53.51% | 4.79 | 1.8389 | 47.65% | 6.29 |
| Jordan | 2.0541 | 39.13% | 7.80 | 2.2143 | 36.10% | 9.16 |

The best validation accuracies observed during training were:

| Model | Best Val Acc | Epoch |
|---|---|---|
| Elman | 47.77% | 14 |
| Jordan | 36.23% | 14 |

The main numerical conclusion is that hidden-state feedback in the Elman model was more effective than output-distribution feedback in the Jordan model under the tested conditions.

## 8. Elman Model Analysis

Elman showed a strong learning trend from the early epochs. Validation accuracy started at 32.13%, reached 47.65% at the end of training, and achieved its best value of 47.77% at epoch 14. Validation loss decreased from 2.4496 to 1.8389, while validation perplexity decreased from 11.58 to 6.29. These reductions indicate that the model progressively learned a more informative probability distribution for the next character.

Training accuracy increased from 26.57% to 53.51%, demonstrating substantial learning on the training data. The gap between training and validation performance was present but not extreme, indicating that the model retained a reasonable degree of generalization to unseen validation sequences.

## 9. Jordan Model Analysis

Jordan also improved during training, but its performance remained weaker than Elman. Validation accuracy increased from 26.18% to 36.10%, with a best value of 36.23% at epoch 14. Validation loss decreased from 2.6422 to 2.2143, and validation perplexity decreased from 14.04 to 9.16.

The results confirm that Jordan learned useful patterns, but more slowly and less effectively. A plausible explanation is its direct use of the previous output as feedback. Prediction errors can therefore influence later recurrent states, which can make optimization and long-range consistency more difficult.

## 10. Analysis of Training Curves

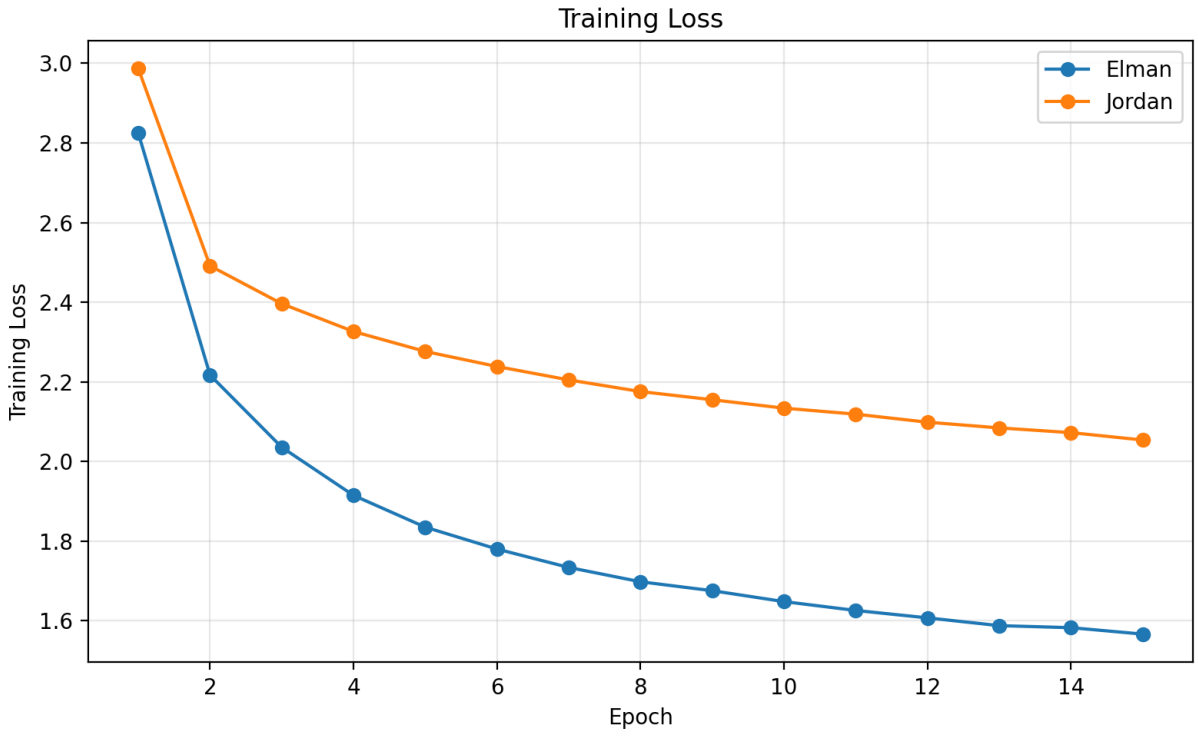


*Figure 1. Training Loss for Elman and Jordan across epochs.*

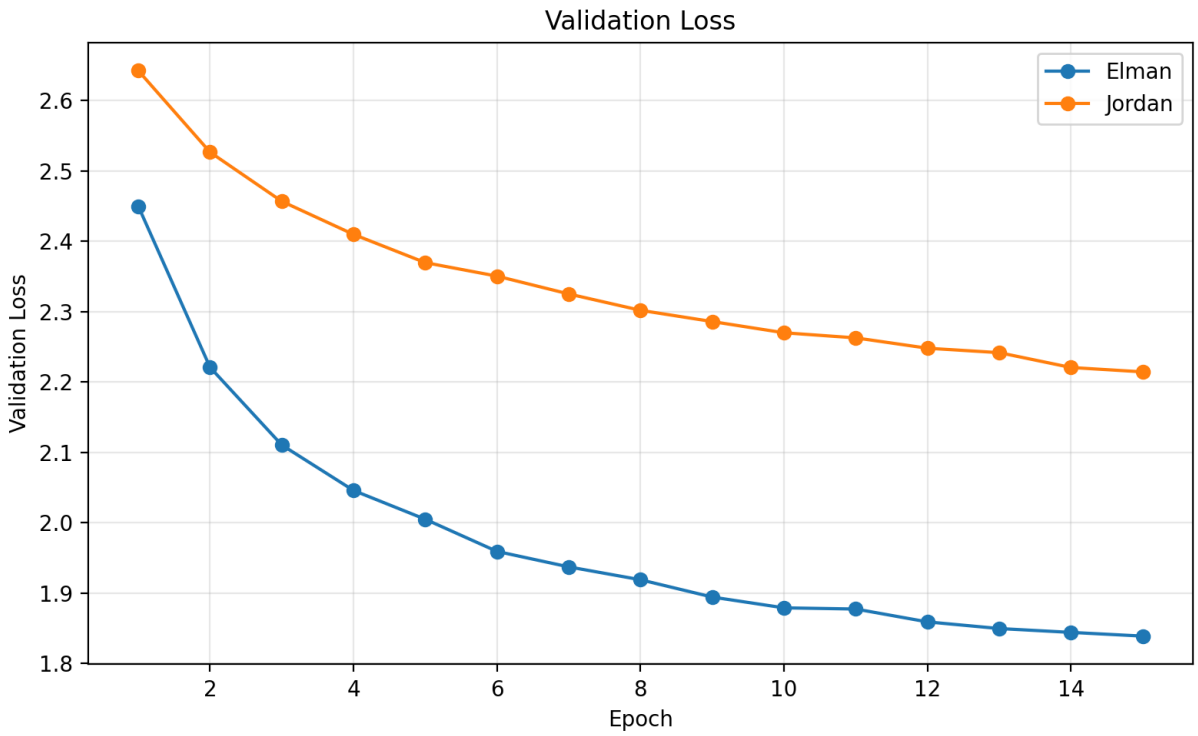


*Figure 2. Validation Loss for Elman and Jordan across epochs.*

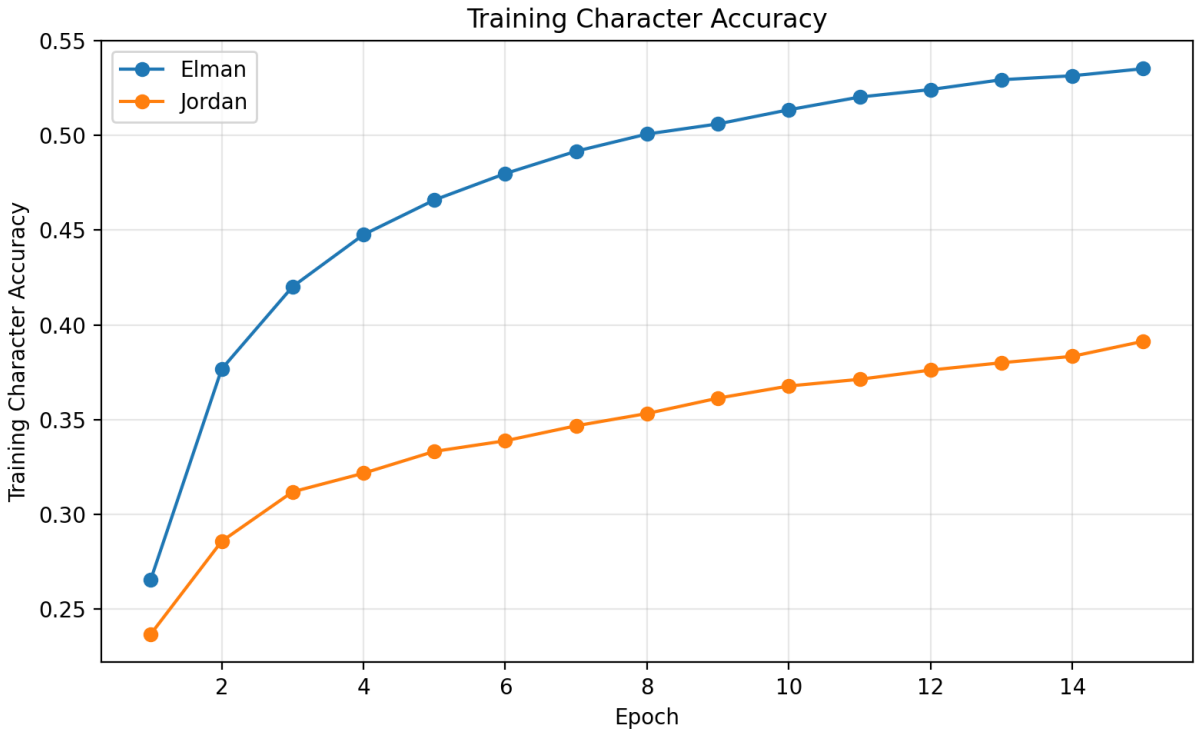


*Figure 3. Training Character Accuracy for Elman and Jordan.*

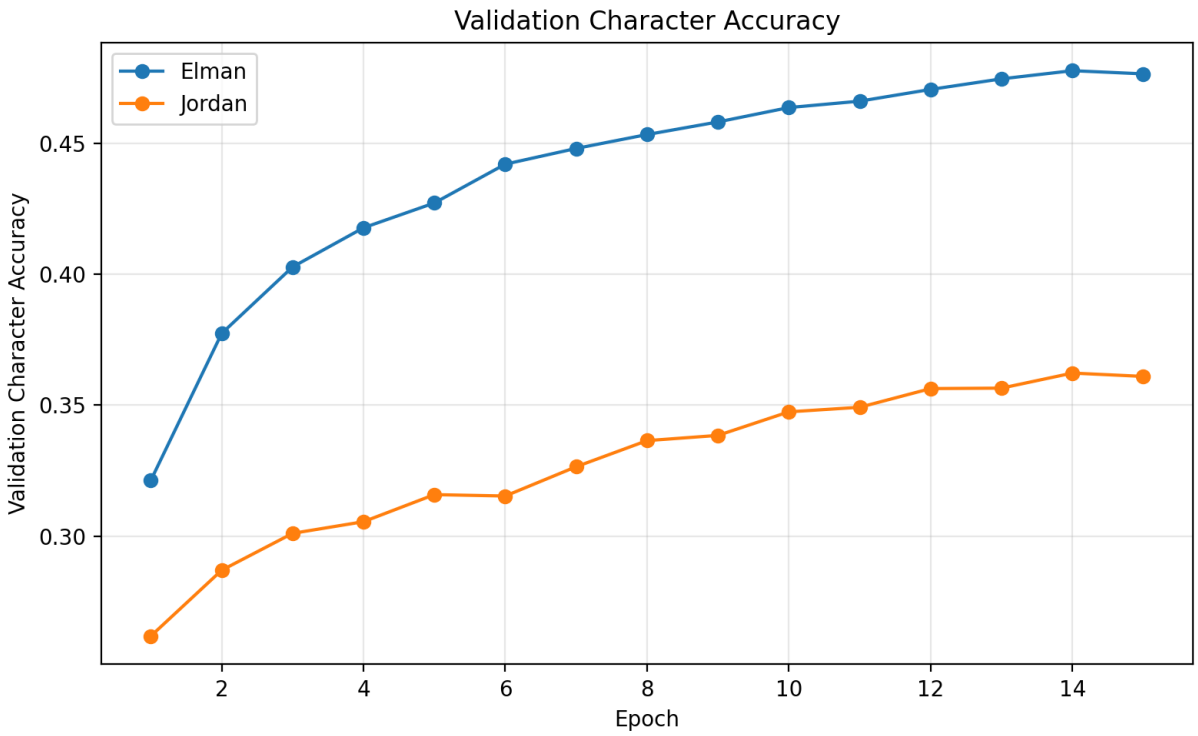


*Figure 4. Validation Character Accuracy for Elman and Jordan.*

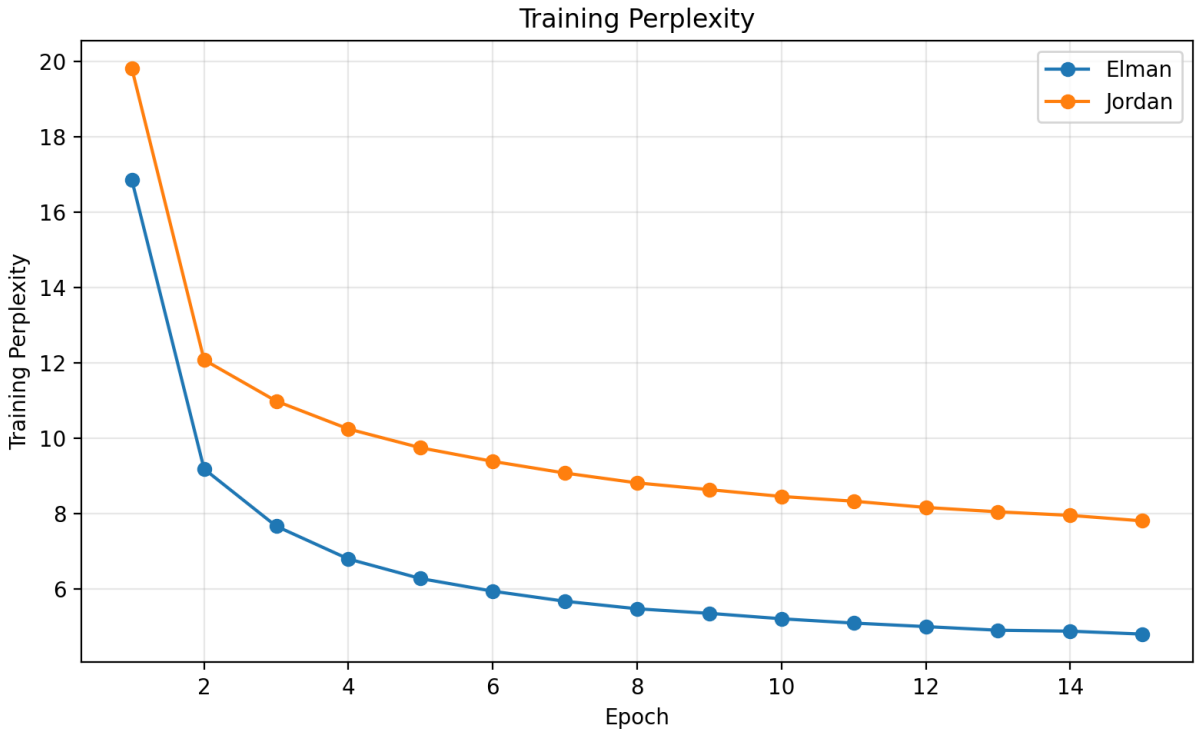


*Figure 5. Training Perplexity for Elman and Jordan.*

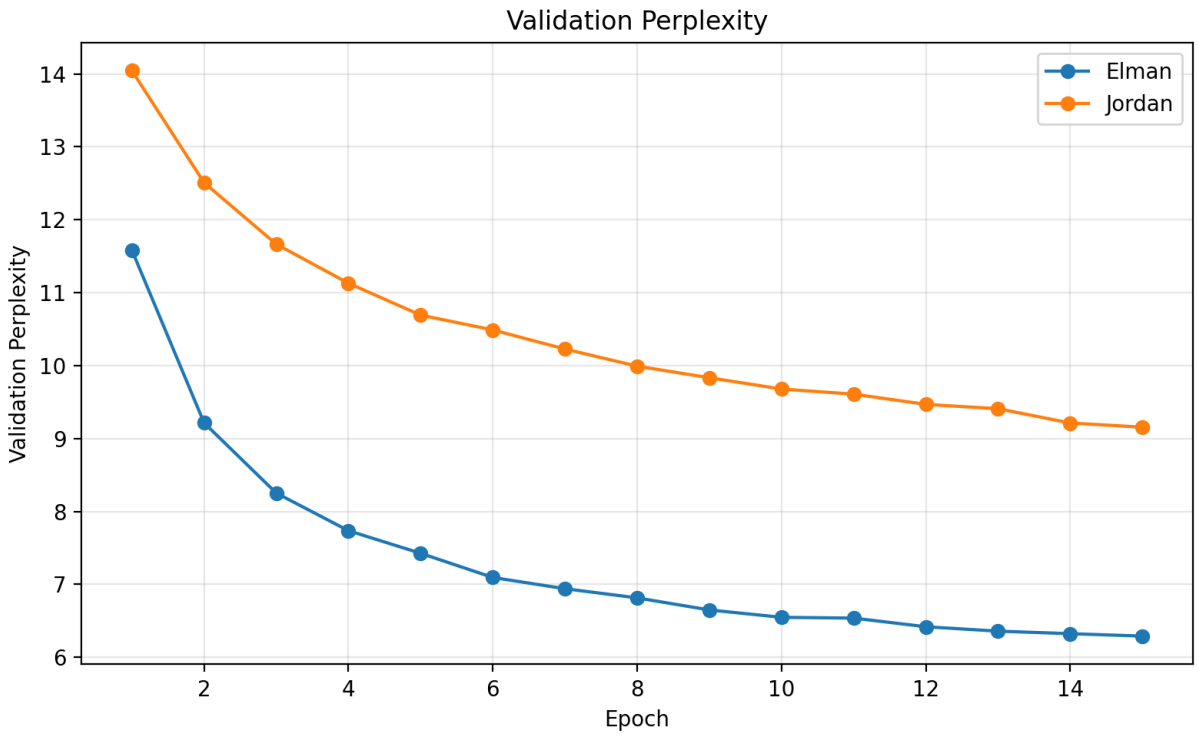


*Figure 6. Validation Perplexity for Elman and Jordan.*

## 10.1 Loss Curves

Training loss decreased for both architectures, confirming that both models learned from the training data. However, Elman decreased faster and reached a substantially lower final value. Elman training loss fell from 2.8245 to 1.5667, whereas Jordan decreased from 2.9865 to 2.0541.

The validation-loss trend was similar. Elman decreased from 2.4496 to 1.8389, while Jordan ended at 2.2143 after starting from 2.6422. This indicates that Elman performed better not only on the training data but also on held-out validation sequences.

## 10.2 Accuracy Curves

Training character accuracy increased more rapidly for Elman. It rose from 26.57% to 53.51%, while Jordan rose from 23.69% to 39.13%. Across the validation epochs, Elman also remained consistently ahead. Elman validation accuracy increased from 32.13% to 47.65%, with a best value of 47.77%; Jordan increased from 26.18% to 36.10%, with a best value of 36.23%.

## 10.3 Perplexity Curves

Perplexity decreased for both models, meaning both became less uncertain about the next character. Elman nevertheless maintained lower perplexity throughout training. Its training perplexity decreased from 16.85 to 4.79 and validation perplexity from 11.58 to 6.29. Jordan decreased from 19.82 to 7.80 on training data and from 14.04 to 9.16 on validation data. The perplexity curves therefore reinforce the conclusion that Elman learned a stronger character-level language model.

# 11. Convergence and Stability Comparison

Considering both the numerical results and the curves, Elman demonstrated faster convergence and better training stability. It reached lower loss and higher accuracy earlier, and its advantage over Jordan remained visible through the final epoch. Jordan showed steady learning, but its progress was slower and its final performance remained clearly below Elman.

| Criterion | Better Model |
|---|---|
| Train Loss | Elman |
| Validation Loss | Elman |
| Train Accuracy | Elman |
| Validation Accuracy | Elman |
| Train Perplexity | Elman |
| Validation Perplexity | Elman |
| Convergence Speed | Elman |
| Stability | Elman |

# 12. Text Generation and the Effect of Temperature

After training, both models generated text from the same seed: “ROMEO:\n”. Three temperatures—0.4, 0.8, and 1.2—were used to study the effect of sampling randomness. Lower temperature makes the model more conservative by favoring high-probability characters, usually improving stability but increasing repetition. Higher temperature increases diversity but also raises the probability of malformed words, noise, and incoherent sequences.

## 12.1 Temperature = 0.4

At 0.4, both models behave conservatively and prefer frequent, high-probability characters. Outputs are therefore more stable than at higher temperatures, although repetition becomes more likely.

Elman: the generated output retained a partial dramatic structure. After the seed, the model produced a character name such as “MENENIUS:” and continued in dialogue-like form. Although malformed words such as “prount”, “meed”, and “manieve” appeared, the overall ordering, character-name pattern, and dialogue structure suggest that Elman learned part of the sequential organization of the corpus. At this temperature, Elman was relatively more regular, readable, and less noisy.

Jordan: the model also attempted to produce a dramatic structure, but the quality was weaker. Names such as “QUS” and “MENENGLARD ICINIUS” were more irregular, and phrases such as “The of the the wer wer the have wis the and the the me” revealed stronger repetition of common tokens. Low temperature stabilized Jordan to some extent, but its output remained more repetitive and less meaningful than Elman.

Comparison: Elman performed better at 0.4 because it preserved dialogue structure more effectively and produced more readable text.

## 12.2 Temperature = 0.8

At 0.8, randomness is higher than at 0.4, allowing more diverse output while usually retaining a reasonable level of coherence. This temperature provided the best balance in the reported experiment.

Elman: the model produced a character name such as “CORIOLANUS:” and continued with dialogue-like text. The output was more diverse and less repetitive than at 0.4. Errors such as the malformed word

“d'trubel” remained, and the language was not fully grammatical, but the preservation of character names and dialogue format indicates a useful balance between diversity and coherence.

Jordan: diversity increased relative to 0.4, but this did not necessarily improve quality. The model continued to produce malformed words, unnatural combinations, and less coherent sentences. Because Jordan feeds its previous output back into the next step, a poor output can influence later states and contribute to continuing instability.

Comparison: Elman was more readable and coherent at 0.8, making this temperature particularly suitable for the Elman model.

## 12.3 Temperature = 1.2

At 1.2, the sampling distribution becomes more permissive and lower-probability characters are selected more often. Diversity increases, but coherence and readability generally decrease.

Elman: the text became more diverse but also more irregular. Spelling errors, incomplete words, and unnatural combinations appeared more frequently. Even so, the hidden-state memory allowed Elman to preserve more of the overall sequential structure than Jordan. Its quality was lower than at 0.8 but still comparatively acceptable.

Jordan: noise and irregularity increased further. High temperature produced more random character choices, and the output-feedback mechanism meant that an irregular prediction could influence later steps. As a result, Jordan suffered a larger loss of coherence than Elman.

Comparison: both models became noisier at 1.2, but the degradation was more severe for Jordan.

## 12.4 Summary of Temperature Effects

| Temperature | Observed Effect |
|---|---|
| 0.4 | More stable and conservative, but more repetitive. |
| 0.8 | Best balance between diversity and coherence in this experiment. |
| 1.2 | Greater diversity, but lower readability and more noise. |

Across all three temperatures, Elman produced more coherent and natural-looking text than Jordan. This qualitative result agrees with the quantitative validation metrics, where Elman achieved lower loss, higher accuracy, and lower perplexity.

## 12.5 Generated Text Samples

**Elman, temperature = 0.4**

```
ROMEO:
MENENIUS:
Shall the stand their good for an and the prount the place, what I state to meed revort,
And show the manieve the mannot hear of the said to see him...
```

**Elman, temperature = 0.8**

```
ROMEO:
CORIOLANUS:
I have not should be the dealt. I at trubel'd of your wan, when have best fin he were up...
```

**Elman, temperature = 1.2**

```
ROMEO:
LARTIUS:
Why, then, the crown'd of breath? I speak no more,
But yet the people, thus in wildness, cry and turn away...
```

**Jordan, temperature = 0.4**

```
ROMEO:
QUS:
MENENGLARD ICINIUS:
CORINGHARD:
The of the the wer wer the have wis the and the the me...
```

**Jordan, temperature = 0.8**

```
ROMEO:
MENENIUS:
The lord the shall in the come and the men,
I have the word and there the state to him...
```

**Jordan, temperature = 1.2**

```
ROMEO:
CAR:
Whar the murd of thee! unot sear, brath'd,
Qenfor no the hame, and his worle to brin...
```

The samples show that Elman is more coherent at low and medium temperatures and better preserves dialogue-like structure and character names. At 0.4, the output is more stable but somewhat repetitive. At 0.8, diversity and readability are better balanced. At 1.2, diversity increases but spelling errors, incomplete words, and irregularity also increase. These problems are more pronounced in Jordan.

## 13. Analytical Summary

The overall results indicate that, for this experiment, Elman was better suited to next-character prediction on Tiny Shakespeare. It achieved higher accuracy, lower loss, and lower perplexity, converged faster, and generated more coherent text. Jordan still learned some textual patterns, but its quantitative performance and generation quality were consistently weaker.

The findings suggest that, for this dataset and these settings, hidden-state feedback is more effective than output-distribution feedback. The feedback mechanism therefore has a significant effect on learning quality, validation performance, and generated-text behavior.

## 14. Final Conclusion

In this project, Elman and Jordan recurrent networks were implemented and compared for character-level next-character prediction on Tiny Shakespeare. Thirty percent of the dataset was used, and both models were trained under identical settings to ensure a fair comparison.

At the end of training, Elman reached 47.65% validation accuracy and 6.29 validation perplexity, whereas Jordan reached 36.10% validation accuracy and 9.16 validation perplexity. The best validation accuracy was 47.77% for Elman and 36.23% for Jordan. These values show a clear advantage for Elman.

The results indicate that using the previous hidden state as memory was more effective for this task than feeding back the previous output distribution. Elman learned the textual patterns more effectively and generalized better to validation data.

The generated-text analysis led to the same conclusion. Elman generally produced more coherent and natural-looking text, while Jordan showed more repetition, irregularity, and malformed structures. Overall, the assignment demonstrates that the choice of recurrent feedback mechanism can substantially affect learning dynamics, validation quality, and the character of generated text.

## Appendix A. Terminal Output from the Actual Run

The following screenshots are taken from the actual execution of p2_completed.py. They show the initial configuration and dataset information, followed by the training output for the Elman and Jordan models. These outputs form the basis of the numerical tables and analyses reported above.

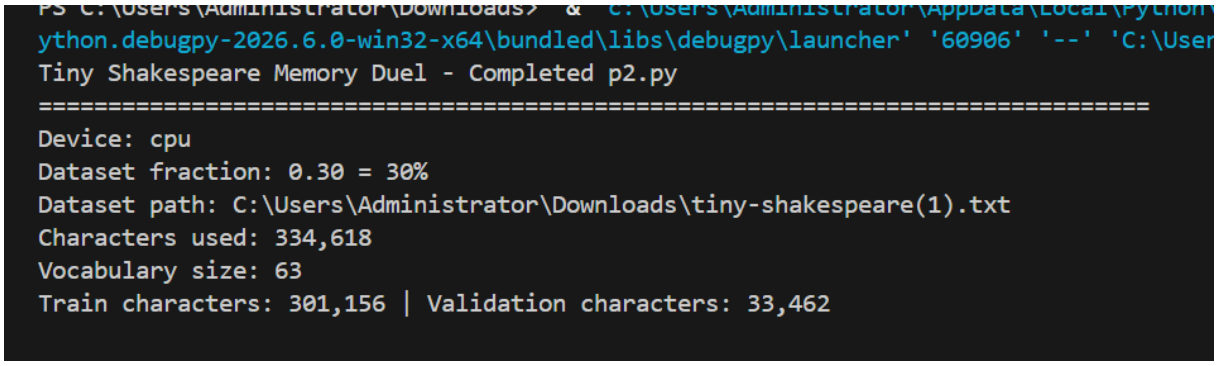


*Figure 7. Initial execution settings and dataset information in the terminal.*

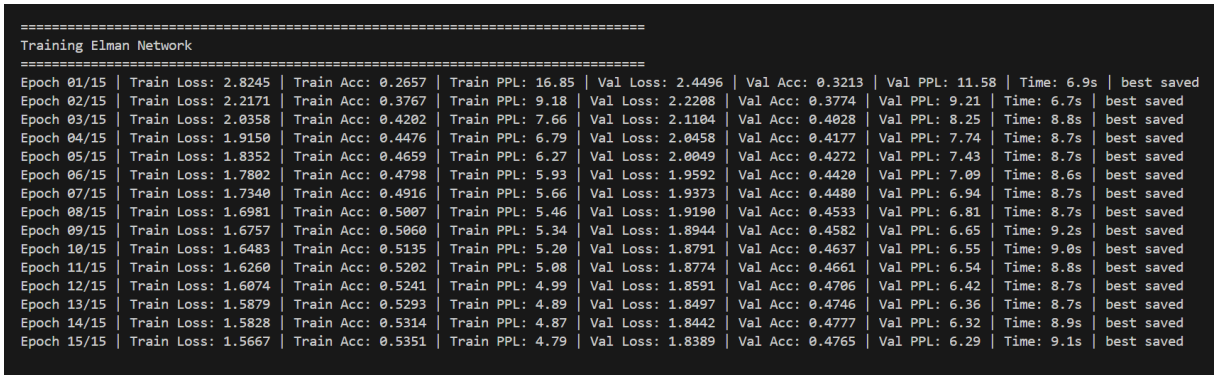


*Figure 8. Elman model training output in the terminal.*

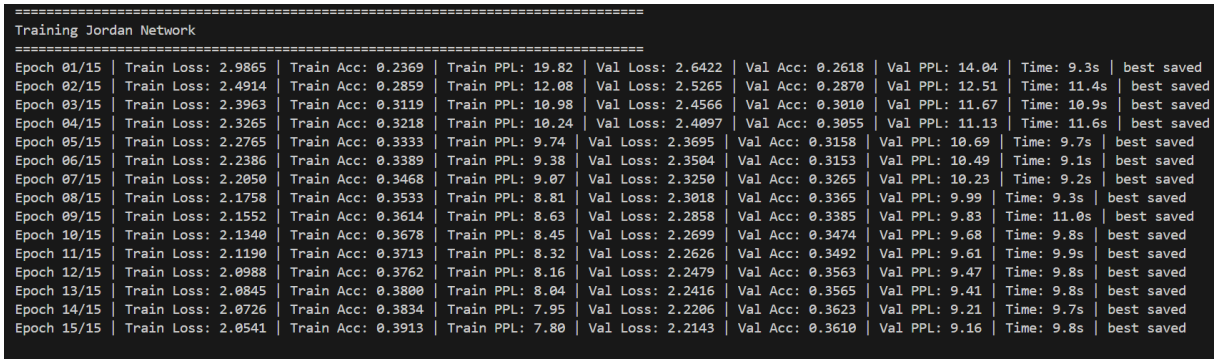


*Figure 9. Jordan model training output in the terminal.*